
```
import threading
from time import sleep

def sleep_sort(i):
    sleep(i)
    global sortedlist
    sortedlist.append(i)
    return(i)

items = [2, 4, 5, 2, 1, 7]
sortedlist = []
ignore_result = [threading.Thread(target = sleep_sort, args = (i,)).start() \
for i in items]
```

The sorted list will be stored in the `sortedlist` variable, and you can ignore the list we create called `ignore_result`. You can see that one advantage of sleep sort is that it can be written concisely in Python. It's also fun to print the `sortedlist` variable before the sorting is done (in this case, within about 7 seconds) because depending on exactly when you execute the `print` command, you'll see a different list. However, sleep sort also has some major disadvantages. One of these is that because it's not possible to sleep for a negative length of time, sleep sort cannot sort lists with negative numbers. Another disadvantage is that sleep sort's execution is highly dependent on outliers—if you append 1,000 to the list, you'll have to wait at least 1,000 seconds for the algorithm to finish executing. Yet another disadvantage is that numbers that are close to each other may be inserted in the wrong order if the threads are not executed perfectly concurrently. Finally, since sleep sort uses threading, it will not be able to execute (well) on hardware or software that does not enable threading (well).

If we had to express sleep sort's runtime in big O notation, we might say that it is $O(\max(list))$. Unlike the runtime of every other well-known sorting algorithm, its runtime depends not on the size of the list but on the size of the elements of the list. This makes sleep sort hard to rely on, because we can only be confident about its performance with certain lists—even a short list may take far too long to sort if any of its elements are too large.

There may never be a practical use for sleep sort, even on a sinking ship. I include it here for a few reasons. First, because it is so different from all other extant sorting algorithms, it reminds us that even the most stale and